Unit No. III

Inheritance and Polymorphism, Exception Handling and Multithreating

Inheritance :-

- Java inheritance is fundamental concept in OOP
- It is the mechanism in java by which one class it is allowed to inherit the features of another class.
- In java inheritance mean creating new classes based on existing class.
- The class that inherit from another class can reuse method & fields of that class.

* Why use inheritance in java ?
→ code reusability.
- Method overiding.
- Abstraction

* Key terminology used in java inheritance

1) Class :-
class is the set of object can share common characteristic/property / Attribute. class is not a real world entities. It is just a template or blueprint from which objects are created.

2) Super class / Parent class
A class whose features are inherited is known super class

TGBPOLYTECHNIC.COM

Scanned with OKEN Scanner

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)

SPPU Engineering & Technical UPDATES
WhatsApp channel

WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b

@SPPU_ENGINEERING_UPDATE

Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

TG:SPPU_BE_COMP

3) Sub class/child class/derived class:-
The class that inherit the other class is known as sub class

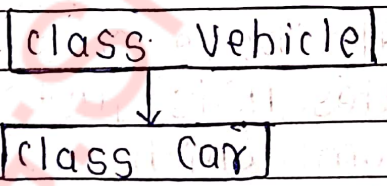
4) Extends keyword
This keyword is used to inherit property from a super class.

Types of inheritance in java

1) Single inheritance
2) Multilevel inheritance
3) Hierarchical Inheritance
4) Multiple Inheritance
5) Hybrid Inheritance.

1) Single Inheritance

- In Single inheritance a sub class is derived from only one super class. It inherits property and character of a single parent class sometimes it is also known as single inheritance.



TG:SPPUBE_COMP

Scanned with OKEN Scanner

```
import java.io.*;
class Vehicle
{
    Vehicle()
    {
        System.out.println("This is Vehicle");
    }
}
class Car extends Vehicle
{
    Car()
    {
        System.out.println("This Vehicle is Car")
    }
}
class Test
{
    public static void main(String[]args)
    {
        Car c = new Car();
        c.Vehicle();
        c.Car();
    }
}
```

OIP = This is Vehicle
This Vehicle is Car.

2) Multilevel Inheritance.

- In multilevel inheritance a derived class will be inheriting a base class & as well as derived class also acts as the base class for other classes

TE-IT-BE-COMP

Scanned with OKEN Scanner

class vehicle. Base class 1

↓

class fourvehiler

↓ Base class 2

class car

Derived class

```
import java.io.*;
class Vehicle
{
    Vehicle()
    {
        System.out.println("This is a vehicle");
    }
}
class Fourwheeler extends Vehicle
{
    Fourwheeler()
    {
        System.out.println("four wheeler vehicle");
    }
}
class Car extends Fourwheeler
{
    Car()
    {
        System.out.println("This 4wheeler vehicle is
                                        car");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Car c = new Car();
        c.Vehicle();
        c.fourwheeler();
        c.car();
    }
}
```

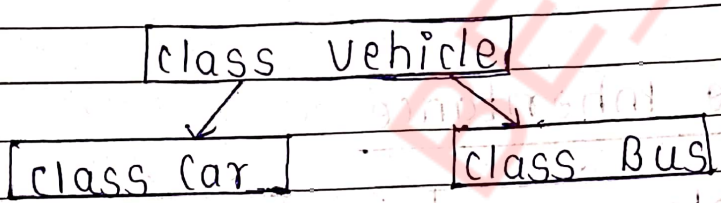
TG:SPPU_TE_BE_COMP

Scanned with OKEN Scanner

O/P = This is vehicle
Four wheeler vehicle
This four wheeler vehicle is car.

3) Hierarchical Inheritance.

In hierarchical inheritance more than one sub class it inherit from single base class ie; more than one class is created from single base class.



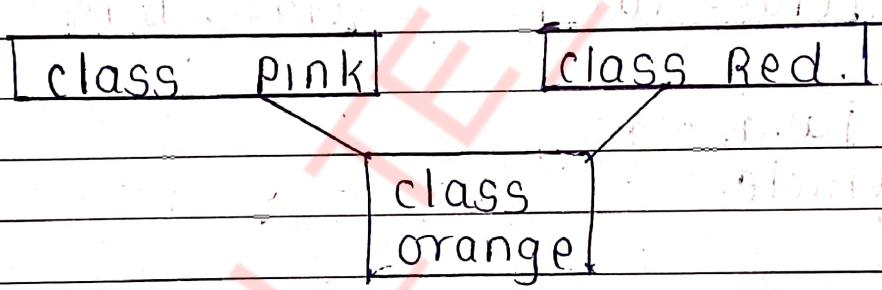

```
import java.io.*;
class Vehicle.
{
    Vehicle()
    {
        System.out.println("This is vehicle");
    }
}
class Car extends Vehicle
{
    Car()
    {
        System.out.println("This vehicle is car"
} }
}
class bus extends car.
{
    bus()
    {
```

TG:SPPU_TE_IT_COMP

Scanned with OKEN Scanner

```
        System.out.println("This is bus");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Car c = new Car();
        Bus b = new Bus();
    }
}
```

4) Multiple Inheritance



TG:SPPU_TE_BE_COMP

Scanned with OKEN Scanner

```
import java.io.*;
class pink.
{
    Pink()
    {
      System.out.println("This colour is pink");
    }
}
class Red.
{
    Red()
    {
      System.out.println("This colour is Red");
    }
}
class orange extends pink, Red
{
    orange()
    {
      System.out.println("This colour is orange")
    }
}
Class Test
{
    public Static void main(string [] args)
    {
      orange o = new orange();
    }
}
```

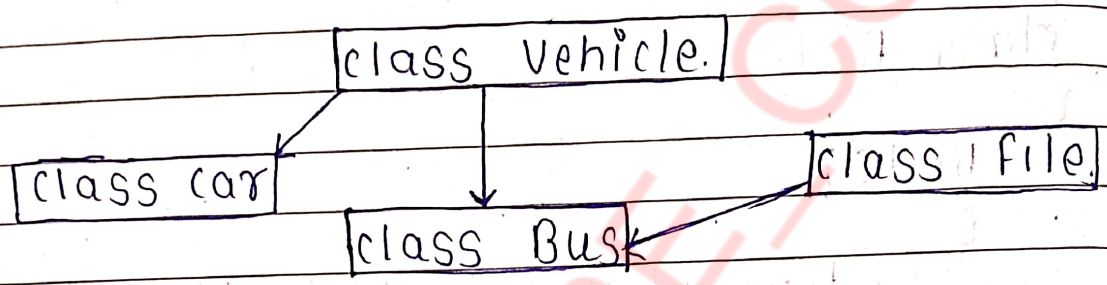
O/P = This colour is pink.
This colour is Red.
This colour is orange.

TC:SPPU-TE-BE-COMP

Scanned with OKEN Scanner

5) Hybrid Inheritance

It is mix of two or more of the above type of inheritance. In java we can achew hybrid inheritance only through interface want multiple inheritance to implement hybrid inheritances




```
import java.io.*;
class Vehicle.
{
    Vehicle()
    {
        System.out.println("This is vehicle");
    }
}
class Car extends Vehicle
{
    Car()
    {
        System.out.println("This is car");
    }
}
class Bus extends Vehicle.
{
    Bus()
    {
        System.out.println("This is bus");
    }
}
```

TG.SPPU-BE-COMP

Scanned with OKEN Scanner

```
class File
{
    File()
    {
we      System.out.println("This is File");
    }
}
Class Bus extends file
{
  Test()
  {
    System.out.println("
  }
}
class Tree
{
   public static void main(String[]args)
   {
     Car c = new car();
     Bus b = new Bus();
     b.Test();
   }
}
```

Advantages of Inheritance

1) Code reusability
2) Abstraction
3) Class hierarchy

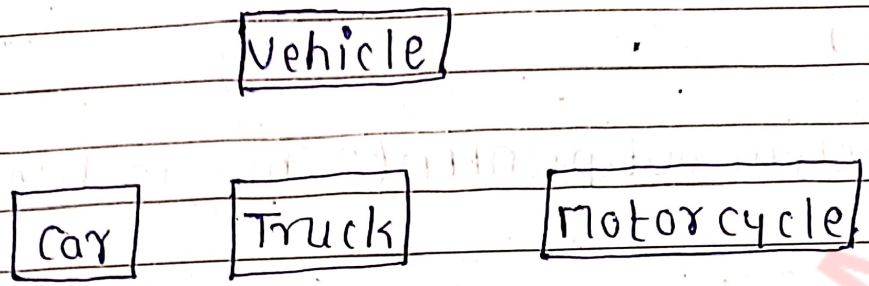
Disadvantage.

1) Complexity
2) Tight compiling.

TECHPUTE-BE-COMP

Scanned with OKEN Scanner

* Super class and Sub class



1) Super class
- A super class is the class from which many sub classes can be created the subclass intents character of super class
- The super class also known as the parent class or Base class.
- In the above Example vehicle is the super class & it subclass are car, truck motor cycle.

- Car, truck & motor cycle are all subclasses of superclass vehicle. They wall inherit common attribute for vehicle such as speed, colour while they have diff. attribute also ie; no. of wheel in car is four while motorcycle is two.

2) Sub class
- A sub class is a class derived from the super class. It inherits the property of the super class & also contain attribute own
- They also know as derived or child class.
- In the above e.g car, truck, motorcycle are subclass & vehicle is super class.

Scanned with OKEN Scanner

Protected Member

In java protected access modifier plays a specific role in managing member visibility within

*Key characteristic of protected member

1) Accessibility within the same package.
- A protected member of a class is accessible by any other class within the same package.

2) Accessibility by sub classes (even in diff. Pack)
- This is the pore aspect of protected in inheritance
- A subclass, regardless of whether it reside in the same pack of superclass or diff. one, can directly access the protected of its super class.

3) Restricted access from one related class from different package
- Classes in the diff. package that are not subclass of class containing the protected that cannot access that member directly.

Scanned with OKEN Scanner

```
import java.io.*;
class sample
{
  protected int year = 2021;
  protected void printYear()
  {
    System.out.println("Its"+year+"!!");
  }
  public static void main(String [] args)
  {
    Sample s = new Sample ();
    System.out.println(s.year);
    s.printYear()
  }
}
```

O/P = Its 2021 !!

* Relationship between superclass & subclass

In java a relationship bet^n superclass & sub class is defined by inhentance this relationship signify "Is a" relationship, where a subclass is type of its super class

1) Super class (Parent / based class)

This is the class whose property and behaviour (method) are inherited by another class

2) Sub class

TO SPRUTEE BE COMP

Scanned with OKEN Scanner

This is the class that inherits from a super class. It extend the super class adding its own unit property & behaviour.

*Key aspects of the relationship

1) Inheritance

The subclass automatically gains access to public & protected member of it super class This promote code reusability & reduce redendancy. (Repetition).

2) Specialization.

The sub class can add fields and method specific to its own functionality, or it can overite inherited method to provide a special implementation.

3) Polymorphism.

Through inhentance a reference variable of super class type can hold and object of it subclass. This allow for flexible & dynamic behaviour, where the specific method called depend on actual objec time & run time

Extends keyword

In java extends keyword used to declar that a class is the subclass of another

Scanned with OKEN Scanner

Single inheritance meaning a class directly inherit only one super class. however, it can implement multiple interface it provide form multiple inheritance.

Object class

- All classes in java implicity or extilicity inherit from the java.lang.Object making it the ultimate super class in java hierarchy
- Object class in java is present in java.lang package.
- Every class is directly or indirectly from object class. if a class does not extend any other class the it is direct child class of the java.object class and its extend another class then it is directly derived.
- The object class directly provide several method such as
  tostring()
  equals()
  hashCode()
- The object class act as the root of the inheritance hierarchy in any java program
- The object class provide multiple which are as follows
  tostring()
  equals()
  hashCode()

Scanned with OKEN Scanner

finalize()
getclass()
clone()
wait()
notify()

```
import java.io.*;
class Person
{
    String n;
    Public Person(String n)
    {
        this.n=n;
    }
    @Override
    public string toString()
    {
        return "Person{name:"+n+"}";
    }
    public static void main(string[] args)
    {
        Person p=new Person("Geek");
        System.out.println(p.toString());
        System.out.println(P.hashCode());
    }
}
```

Polymorphism

- polymorphism in java derived from Greek word Poly = many & morphs = forms
- It is fundamental concept in oop that enable to single entities to take on

TG-SPIDER BE-COMP

Scanned with OKEN Scanner

. multiple forms or to perform a single
It allow object of diff. class to be
treated of object of a common type
typically there superclass and the
interface unit of implement

* Abstract class & methods

- In java Abstract classes & methods are fundamental concept for acheiving abstraction & defining blueprint for sub class.

* Abstract class

- An abstract class is a class that is declared with the abstract keyword. It cannot be instantiated directly, meaning you cannot create object of an abstract class using new
- Abstract classes can contain a mix of concrete (implemented method) and abstra (unimplemented) methods.
- They can also have member variable, constructor and static methods.
- An Abstract class serve based or other classes to extend, providing common functionality & enforcing structure for it class

* Abstract method

An abstract is a method declare within

an abstract class using the abstract keyword.

- It does not have a method body or implementation, it is simply by declaration followed by semicolon.
- It must be implement any concrete (non abstract) subclass that extends abstract class.
- If a (sub class does not implement or inherited abstract method it most also declared as abstract.

```
import java.io.*;
abstract class Sunstar
{
  abstract void printInfo();
}
class Employee extends Sunstar
{
void printInfo()
{
  String name = "Sahyadri";
  int age = 21
  Float salary = 22.2F;
  System.out.println(name);
  System.out.println(age);
  System.out.println(salary);
}
}
class Base
{
public static void main(String[]args)
{
  Sunstar S = new Employee
  Employee();
```

final keyword:

- In java 'final' keyword is a non access modifier used to restrict modification or extension of variable, method and classes.

* final class

- A final class cannot be extended or inherited by any other class.
- This means no subclass can be created from a final class.
- This is often used to creat immutable classes like java.lang.String, where the state of object cannot be change after creation, ensuring data integrating & thread safety.

* final method

- A final method cannot be overridden by any subclass
- Ones a method is declare as final in super class it implementation is consider complete & unchangeable.
- This is useful for preventing sub class for altering critical behaviour our ensuring consistant functionality across hierarchy

Scanned with OKEN Scanner

```
import java.io.*;
class Parent
{
    public final void display()
    {
        System.out.println("This is final method");
    }
}
class Child extends Parent
{
    public void display()
    {
        System.out.println("Attempting to override");
    }
}
class Demo
{
    public static void main(String[] args)
    {
        Child c = new Child();
        c.display();
    }
}
```

Dynamic binding

- Dynamic binding in java also known as late binding or run time polymorphism refere to the process where the JVM determines which specific method implementated to invoke based on actual type of object rather than its declare type.

Scanned with OKEN Scanner

Dynamic binding primarily occurs in java through;

1) Method overiding

- When a subclass provide its own implementation for method already define it super class & at object of subclass is refered to by super class type variable. The overiden method in sub class is called as routins.
- The JVM identify the actual object type at execution and dispatches are to the appropriate overridden method.

```
import java.io.*;
class Animal
{
  void makeSound()
  {
    System.out.println("Animal makes a sound");
  }
}
class Cat extends Animal
{
  void makeSound()
  {
    System.out.println("Cat makes a sound");
  }
}
```

Scanned with OKEN Scanner

```
class Demo
{
  public static void main(String[] args)
  {
    Animal a = new Cat(); // declared type is
    Animal, actual type is Cat.
    a.makeSound(); // Calls Cat makeSound()
    give to dynamic binding.
  }
}
```

In this example even the a is declared as an animal type because it hold s a cat object, makesound() method of the

* Key characteristic & Benefits

1) Run time resolution ~~& benefits~~

The method to be executed is determ during program execution, not during Compilation

2) Polymorphism

It enables polymorphism allowing object take on many form & response diff to the same method call based on the actual type

3) Flexibily & extensibility.

Scanned with OKEN Scanner

Dynamic binding promote flexible & extensible by allowing new sub class to added modifing exiting code that uses superclass references

4) Used for overidden method.

It is specifically used for method that are overiden in sub classes

Interfaces

- Interfaces in java is a blueprint of class.
- It can contain method signature (Abstract method) and default method, static method & constant variable. Interfaces are used to acheive abstraction and multiple inheritance.
- Interface is completely abstract type that can contain only abstract method. (methods without the body)

* key characteristic of java interface

1) Abstraction
2) Multiple Inheritance
3) Constance
4) Abstract method
5) Implement keyword
6) extends keyword.
7) Default method.

Scanned with OKEN Scanner

Example

```
interface Walkable
{
  void walk();
}
interface Swimmable
{
  void swim();
}
class Duck implements Walkable, Swimmable
{
  public void walk()
  {
    System.out.println("Duck is Walking");
  }
  public void swim()
  {
    System.out.println("Duck is swimming");
  }
}
class Demo
{
  public static void main(String[] args)
  {
    Duck d = new Duck();
    d.walk();
    d.swim();
  }
}
```

TG-SPPU TE-BE-COMP

Scanned with OKEN Scanner

Exception Handling

- Exception handling in java provides a robust mechanism to deel with run time error and maintain flow of the program.

#Fundamental of exception handling

1) try
This block encloses the code that might potentialy through an exception.

2) catch.
This block immediately follows a try block and contains code to handle a specific type of exception that might to be thrw with in try block.

3) finally
This block follows try & catch block and contains code that will always execute regardless of whether & exception occur or was handle

4) throw
This keyword is used to explicity thrw & exception object from a method or block of code.

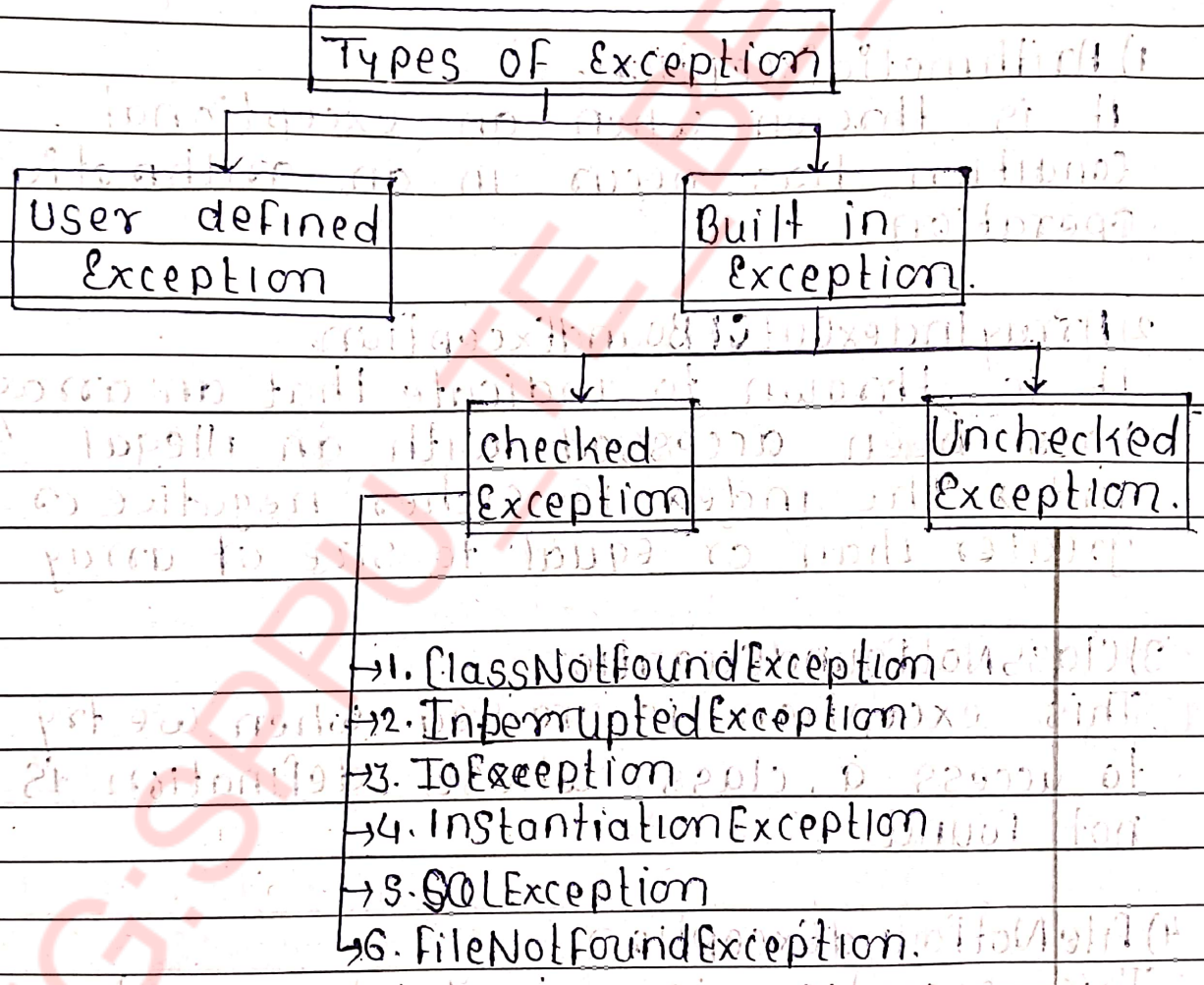
5) throws
This keyword is used in a method Signature to declare that the method

Scanned with OKEN Scanner

might throw one or specified check exception.

*Types of exception handling.

Java define several type of exception that relate to it various type library java also allows user to define there own exception



*Unchecked Exception.

1. ClassCastException
2. ~~class~~
2. ArithmeticException
3. NullPointerException.

Scanned with OKEN Scanner

4. ArrayIndexOutBoundsException.
5. ArrayStoreException.
6. IllegalThreadStateException.

* Built in Exception

- Built in Exception are the exception that are available in java libraries this expression suitable for certain error are situation.

1) ArithmeticException.
It is thrown when an exceptional condition has occur in an arithmatic operation.

2) ArrayIndexOutOfBoundException.
It is thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to size of array

3) ClassNotFoundException.
This exception is raised when we try to access a class whose defination is not found.

4) FileNotFoundException.
This exception is raised when a file is not accessible or does not open.

5) IoException
It is thrown when an input output opera^{r}

fail or interupted

6) InterruptedException.
In is thrown when a thread is waiting, slipping, doing some processing and it is interrupted

7) NoSuchFieldException.
It is thrown when a class does not contain the filled (or variable) specified

8) NoSuchMethodException
It is thrown when accessing method that is not found.

9) NullPointerException
x This exception is raised when method
- This exception is raised when referring the member of a null object. Null represents nothing.

10) NumberPointerException
This exception is raised when method could not convert a String into a numeric format

11) RuntimeException
This represent an exception that occurs during runtime.

12) StringIndexOutOfBoundsException
It is thrown by string class method to indicate that an index is either negative

or greater than size of string.

13) IllegalArgumentException.
- This exception will throw the error or error statement when the method receive an argument which is accurately fit to given relation or condition.

#User defined. Exception

- In java a user defined exception also known as custom exception.
- Is a class created by a programmer to handle specific error condition or business logic violation with in an application
- This exception are not part of java built in exception hierarchy but are designed to more precise error handling to the application unique requirement.

* Try + catch

- The try keyword in java used to define a block of code that will be tested for exception while it is being executed
- The try block is usually followed by one or more catch blocks which handle specific exception that may be thrown within try block.
- Optionally a finally block can be used to execute code regardless of whether & exception was thrown or not

TO BE PUBLISHED BY ICON

Scanned with OKEN Scanner

Syntax:-

```
try
{
   //code that may throw an exception
}
catch(ExceptionType1 e1)
{
   // Code to handle ExceptionType1
}
finally
{
   //code that will always execute
}
```

e.g:-

```
import java.io.*;
public class Demo
{
   public static void main(String[] args)
   {
      try
      {
         int division = 10/0;
      }
      catch (ArithmeticException e)
      {
         System.out.println("Arithmetic
         Exception caught "; e.getmessage())
      }
   }
}
```

TG:SPJ_TE_BE_COMP

Scanned with OKEN Scanner

#Multiple try-catch block.

- multiple catch blocks can java are used to catch Slash handle multiple excepn that may be thrown from a particular code section
- The try block can have multiple catch block to handle multiple exception.

Syntax:-

```
try
{
    // try block
}
catch(ExceptionType1 e1)
{
    // catch block
}
catch(ExceptionType2 e2)
{
    // catch block.
}
catch(ExceptionType3 e3)
{
    // catch block.
}
```

TGSPUTE-BE-ONLY

Scanned with OKEN Scanner

```
import java.io.*;
import java.util.scanner;
Public class Test
{
  public static void main(String[] args)
  {
    Scanner scn = new Scanner(system.in);
    try
    {
      int n = Integer.phaseInt(Scn.nextline());
      if(99%n==0)
      {
        System.out.println(n+" is factor of 99");
      }
      catch (ArithmicException e)
      {
        System.out.println("Arithmetic Excepn"
                           +e);
      }
      catch(NumberformatException e)
      {
        System.out.println("Exception occur="
                           +e);
      }
    }
  }
```

Nested try Statements

- A nested try block in java refers to try block placed inside another try block.
- This structure is used in exception hand-

TG:SPPU_TE_COMP

Scanned with OKEN Scanner

· dling to manage exception & diff. levels of program execution or when diff. part of code block might throw distinct type of exception.

Syntax:-

```
try
{
   // outer try block
   try
   {
      // inner try block.
   }
   catch
   {
      //Handle inner Exception
   }
}
catch
{
   // Handle outer Exception
}
```

TG:SPPU-TE-BE-COMP

Scanned with OKEN Scanner

```
import java.io.*;
class Team
{
   public static void main(string[] args)
   {
   try
     {
        int a[]={1,2,3,4,5}
        System.out.println(a[5]);
        try
        {
           int de=a[2]/0
        }
        catch(ArithmeticException e1)
        {
           System.out.println("Division by zero
            is not possible");
        }
     }
     catch(ArrayIndexOutOfBoundException e2)
     {
        System.out.println("Element at such
        index does not exist");
     }
   }
}
```

* Uses cases

1) Handling Exception at different scopes
- When a method or blocks of code condense operations that can fail independently & you want handle distinct

TO-SPOT-THE-RE-COMP

Scanned with OKEN Scanner

2) Resource management
Although try with resources is often preferred for resources management, nested try block can be used to ensure that cleanup occur during the primary operation even if an exception occur during the primary operation.

*Java throw

The throw keyword in java is used to explicitly through an exception from a method or any block of code.
We can throw either checked or unchecked exception.
The throw keyword is mainly used to throw custom exception.

```
import java.io.*;
class Red
{
  public static void main (String [] args)
  {
    int numerator = 1;
    int denominator = 0;
    if (denominator == 0)
    {
      throw new ArithmeticException
      ("cannot divide by zero");
    }
    else
    {
      System.out.println (Numerator/denom)
    }
  }
}
```

TG.OBULTEBE.COM

Scanned with OKEN Scanner

* Java throws

- throws is keyword in java that is used in method signature to indicate that this method might throw one of the listed type exception.
- The collar to this method has to handle the exception using a try-catch block.

* Syntax :-

type method_name (parameters) throws exception_list

Where, exception_list is a comma separated list of all exceptions which a method might throws

Q:-
```
import java.io.*;
class Pink
{
    static void fun()throws IllegalAccessException
    {
        System.out.println("Inside fun()");
        throws new IllegalAccessException("demo");
    }
    public static void main(String[] args)
    {
        try
        {
            fun();
        }
        catch(IllegalAccessExeeption e)
        {
            System.out.println("caught in main");
        }
```

* Difference between throw & throws

throw	throws
1) It is used explicitly throw an exception.	It is used to declare that the method might throw one or more exception.
2) It is used inside a block of code	It is used in the method signature
3) It can throw both check and unchecked exception	It is only used for check exception. unchecked exception do not required throws
4) The method or block throws the exception	The method collar responsible for handling the exception
5) Stops the current flow of the execution immediately	It forces the collar to handle the declar exceptions.
6) throw new ArithmeticException("Error")	public void myMethod() throws IoException{}

* finally

finally block in java is an integral part

of exception handling used in conjuction with try and catch class.

- It's primary purpose is to ensure that a specific block of code is always executed, regardless of whether & excepn occur in the try block or is caught by a catch block.

* Syntax :-

```
try
{
    // code that might throw an exception
}
catch
{
    // code to handle the exception
}
finally
{
    // code that will always execute..
}
```

TO-SPPU-FREE-BE-COMP

Scanned with OKEN Scanner

```
import java.io.*;
class Demo
{
  public static void main(String[] args)
  {
    try
    {
      System.out.println("Inside try block");
      System.out.println(3412);
    }
    catch(ArithmeticException e)
    {
      System.out.println("ArithmeticException");
    }
    finally
    {
      System.out.println("finally I execute");
    }
  }
}
```

characteristic finally

1) guaranted execution :- The code with in the finally block is guarranted to execute, irrespective to whether and exception throw caught or not throw at all

2) Resource Management.
- It is commonly used for releasing resources that have been acquired such as closing file stream, data base connection

Scanned with OKEN Scanner

Multithreading.

- Multithreading in java is a process of execute multiple.
- It is a java feature that lets a pro run multiple thread at the same time so task work in parallel and use CPU more efficiently.
- In java multithreading is concurrent execun of two or more thread to maximize the ultilization of the CPU.
- In a single core environment java multithreading manage by JVM and the OS.
- It is mostly used in game and animation

* Advantage.

1) It does not block the user because threads are independent & you can perform multiple operation at same time.

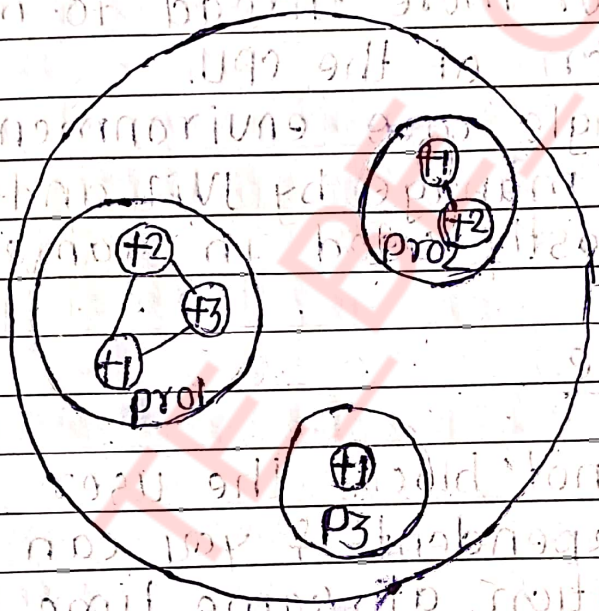
2) You can perform many operation to whether so it same time.

3) Threads are independent so it does not effect any thread if an exception occur Single thread.

Thread.

- A thread is a light wet process the smallest unit of processing
- It is separate path of execution theead

are independent. If there occurl exception in one thread it does not affect other thre
It uses a shared memory area.
- When a java program start one thred begin running immediately which is ealled main thread. This thread respon- sible for executing the main threads of a program.



- As shown above fig. a thread is executed inside the process there can multiple content swiching between them
- There can multiple process inside OS and one process can have multiple threads at a time one thread is executed only

#Creating thread in java

In java, a thread is lightweight process every java program executes by thread

called the main thread.
- When a java program get executed, the main method create automatically. All other threads called from main method
- The java programming language provides two method to create thread and they are listed below.
- Using thread class (by extending thread class) Using Runnable interface)
- lets see how to create threads using each of above

* Extending thread class.

- The java contains a built in class thread inside the java.lang package.
- The thread class contains all the methods that are related to the threads
- To create a thread using thread, class, following the step given below.

Step 1:-
Create a class as a child of thread class. That means, create a class that extends thread class.

Step 2:-
Override the run() method with the code i.e; to be executed by the thread The run() method must be public while ovrriding

Step 3:-

Create the object of the newly create class in main() method.

step 4:-
call the start() method on the object created in the above step.

e.g:- import java.io.*;
class Sample extends Thread
{
public void run()
{
System.out.println("Thread is under Running");
for (int i=1; i<=10; i++)
{
System.out.println("i=" +i);
}
}
public Demo
{
public static void main(String[] args)
{
SampleThread t1 = new SampleThread();
System.out.println("Thread about to start");
t1.start();
}
}

*Main thread

In java the main thread is fundamental

component of every java application
- It is the initial thread that the Java virtual machine (JVM) automatically create and starts when a java program being execution

* Key characteristic of main method

1) Entry point
The main method is responsible for executing the main() method of your java program which serves as the program entry point.

2) Parent Thread
If often act as the "parent" thread from which offer "child" thread can be swaned or created, enabling concurrent execution

3) Foreground Thread
The main thread is a foreground thread meaning the java application will not a terminate until the main thread complete its executions

4) Control and manipulation
- While automatically created, the main thread can be controlled and manipulate like any other thread object.
- You can obtain the interface to the main thread using thread.currentThread{} and the use method like setName{}; setPriority() or sleep() to manage its behaviour.

Scanned with OKEN Scanner

```
import java.io.*;
package com.tutorialspoint;
public class TestThread
{
   public void printInfo()
   {
      System.out.println("Thread Name:" +
      Thread.currentThread().getName())
      System.out.println("Thread priority:" +
      Thread.currentThread.get priority());
   }
   public static void main(String[] args)
   {
      TestThread thread = new TestThread();
      thread.printInfo();
   }
}
```

* Java Thread class:

- Java provides Thread class to achieve thread programming.
- Thread class provides constructors and methods to create and perform operation on thread.
- Thread class extends object class and implements Runnable interface.

* Life cycle of a thread

- A thread goes through various stages in its life cycle. for example: a thread is born, started, runs and then dies. The

Scanned with OKEN Scanner

following diagram shows the complete life cycle of a thread.

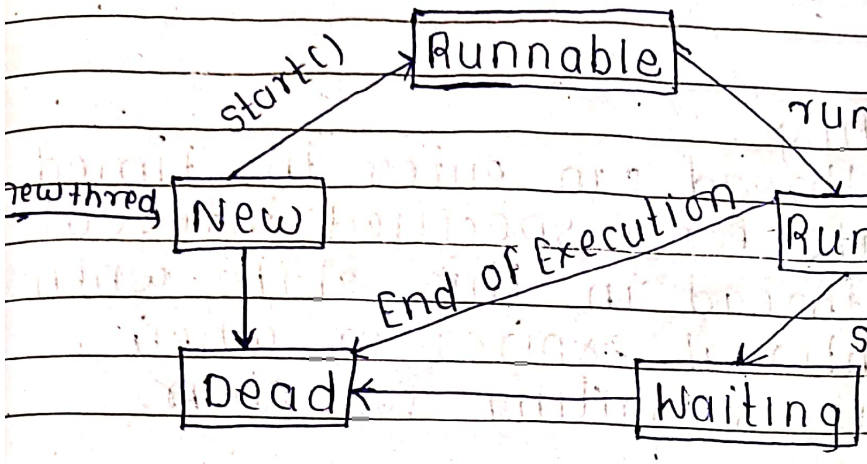


Fig:- life cycle of thread

* Following are the stages of the life cycle thread.

1) New :- A new thread begins its life cycle in the new stage. It remains in this state until the program starts the thread. It is also referred to as a born thread.

2) Runnable :-
After a newly born thread is started the thread becomes runnable. A thread in this state is considered to the be executing its block.

3) Waiting :-
Sometimes, a thread transition to the waiting state while the thread wait for another thread to perform a task A thread transition back to the runnable.

state only when another thread signals the waiting thread to continue executing.

4) Timed Waiting
A runable thread can enter the timed waiting state for a specified interval of time. A thread in this state when that time interval expires or when the event it is waiting for occur.

5) Terminated (Dead):
A runnable thread enters the terminat state when it completes its task or otherwise terminates

* Java thread method

Sr. No	Modifier & type	Method	Description
1)	void	start()	It is used to start the execution of the thread
2)	void	run()	It is used to do an
3)	static void	sleep()	It sleep a thread for the specified amou of time
4)	static thread	currentT-hread()	It returns a referen to the currently execu-ting thread object

TGSPPU-TE-BE-COMP

Scanned with OKEN Scanner